04 kadanes

Kadane’s Algorithm

1. What is it, and when is it used?

Kadane’s Algorithm finds the **maximum sum contiguous subarray** in an array in a single O(n) pass. It’s used whenever you need the best-value contiguous slice of an array under a sum-based objective — maximum, and with light modification, minimum.

2. Why is it used?

Brute force checks all $O(n^2)$ subarrays (or $O(n^3)$ if you recompute sums naively). Kadane’s collapses this to **O(n)** using the simple observation that a negative running sum can never help a future subarray, so you reset it to zero (or to the current element) whenever it goes negative.

3. Where is it used?

- Maximum subarray sum / maximum sum circular subarray
- Maximum product subarray (variant tracking max AND min because of sign flips)
- Best time to buy/sell stock (max difference = a Kadane’s variant)
- Maximum sum rectangle in a 2D matrix (Kadane’s applied row-by-row after column compression)
- House Robber-style DP problems (related “running best” idea)
- Financial/data analysis: longest profitable streak, max gain in a time series

4. How do you identify it’s a Kadane’s problem?

Signal phrases (2-minute test): - “maximum sum subarray” / “maximum sum contiguous subarray” - “maximum product subarray” - “best time to buy and sell stock” (single transaction) - “maximum sum circular subarray” - “largest sum of a contiguous sub-list/sub-range” - Array contains **negative numbers** and you need the best contiguous run (if all positive, it’s usually trivial — whole array or sliding window instead)

5. Can you identify it from constraints?

Constraint clue	Implication
Array has both positive & negative integers	Kadane’s needed (can’t just take whole array)
“Contiguous subarray” explicitly stated	Kadane’s, not subsequence (subsequence would be different DP)
n up to $10^{5–10}6$, need O(n)	Kadane’s fits perfectly
Product instead of sum	Kadane’s variant tracking both max and min running product
Array is circular (wraps around)	Kadane’s + (totalSum − minSubarraySum) trick
2D matrix, “max sum sub-rectangle”	Kadane’s nested inside row-compression loops, $O(rows^2 \times cols)$

6. Types / Variants

1. **Classic Kadane’s (Max Sum Subarray)** — track `currentMax` and `globalMax`.
2. **Kadane’s for Minimum Subarray Sum** — same logic, flipped (or negate array and reuse max version).
3. **Max Product Subarray** — must track both running max and running min (a negative number can turn the smallest product into the largest).

4. **Max Sum Circular Subarray** — `max(Kadane(arr), totalSum - Kadane_min(arr))`, handling the all-negative edge case.
5. **2D Kadane's** — fix a pair of rows, compress into a 1D array of column sums, run classic Kadane's on it; repeat for all row pairs.
6. **Kadane's with index tracking** — also return the start/end indices of the best subarray, not just the sum.

7. Rationale / Intuition

At each index `i`, you ask: *"Is it better to extend the previous subarray by including `arr[i]`, or to start a brand-new subarray at `i`?"* Formally: `currentMax = max(arr[i], currentMax + arr[i])`. This is because a prefix with negative sum can only drag down any suffix appended to it — so once `currentMax` turns negative, you're strictly better off "restarting." This is really a **1D dynamic programming** recurrence in disguise, where the state is "best subarray ending exactly at index i."

8. Time & Space Complexity

Variant	Time	Space
Classic Kadane's	O(n)	O(1)
Max product subarray	O(n)	O(1)
Max sum circular subarray	O(n)	O(1)
2D Kadane's (matrix)	O(rows² × cols)	O(cols)

9. Comparison with Other Patterns

Pattern	Handles	Complexity	Key difference
Kadane's	Max/min sum of a *contiguous* subarray	O(n)	DP with O(1) state, reset-on-negative
Prefix Sum	Range sum queries, count subarrays == K	O(n) build / O(1) query	Doesn't directly optimize; used for lookup
Sliding Window	Max/min of *fixed-size or variable-size* window under sum/count constraint, **requires all-positive** or monotonic property	O(n)	Needs monotonic shrink/grow condition; Kadane's works with negatives
DP (general)	Broader optimization with more complex state	Varies	Kadane's is literally the simplest 1-state DP

Key distinguishing test: if the array **can contain negatives** and you want the best contiguous run, that's Kadane's. If the array is **all non-negative** and you want a sum/count constraint window, that's usually sliding window instead.

10. Reusable Java Templates

10.1 Classic Kadane's — Maximum Subarray Sum

```java
class Kadane {
    public int maxSubArray(int[] nums) {
        int currentMax = nums[0];
        int globalMax = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentMax = Math.max(nums[i], currentMax + nums[i]);
            globalMax = Math.max(globalMax, currentMax);
        }
        return globalMax;
    }
```

```
    }
```

10.2 Kadane’s with Start/End Index Tracking

```
class KadaneWithIndices {
    public int[] maxSubArrayIndices(int[] nums) {
        int currentMax = nums[0], globalMax = nums[0];
        int start = 0, bestStart = 0, bestEnd = 0;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > currentMax + nums[i]) {
                currentMax = nums[i];
                start = i;
            } else {
                currentMax += nums[i];
            }
            if (currentMax > globalMax) {
                globalMax = currentMax;
                bestStart = start;
                bestEnd = i;
            }
        }
        return new int[]{bestStart, bestEnd, globalMax};
    }
}
```

10.3 Maximum Product Subarray

```
class MaxProductSubarray {
    public int maxProduct(int[] nums) {
        int maxProd = nums[0], minProd = nums[0], result = nums[0];

        for (int i = 1; i < nums.length; i++) {
            int curr = nums[i];
            if (curr < 0) {
                int temp = maxProd;
                maxProd = minProd;
                minProd = temp;
            }
            maxProd = Math.max(curr, maxProd * curr);
            minProd = Math.min(curr, minProd * curr);
            result = Math.max(result, maxProd);
        }
        return result;
    }
}
```

10.4 Maximum Sum Circular Subarray

```
class MaxSumCircularSubarray {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0, currMax = 0, maxSum = nums[0], currMin = 0, minSum = nums[0];

        for (int num : nums) {
            currMax = Math.max(currMax + num, num);
            maxSum = Math.max(maxSum, currMax);

            currMin = Math.min(currMin + num, num);
            minSum = Math.min(minSum, currMin);

            total += num;
        }
```

```
            if (maxSum < 0) return maxSum; // all negative edge case
            return Math.max(maxSum, total - minSum);
        }
    }
```

11. Quick Recognition Checklist (2-minute test)

- ☐ Array has negative numbers?
- ☐ Asked for “contiguous” subarray max/min sum or product?
- ☐ O(n) required, brute force O(n²) too slow?
- ☐ Not asking about a fixed-size window or a count/frequency constraint (that’s sliding window territory)?

If 3+ checked → Kadane’s Algorithm.